

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет об исследовательском проекте на тему:

**Применение обучения с подкреплением
в исследовании игр в автобаттлерах**

Выполнил:
студент группы БПМИ243
Пономарев Тимур Евгеньевич

Приняли руководители проекта:
к.ф.-м.н.
Кудеров Пётр Викторович

Старший преподаватель
Департамента больших данных
и информационного поиска
Горденко Мария Константиновна

Москва 2026

Содержание

Аннотация	3
1 Введение	4
1.1 Предметная область	4
1.2 Постановка задачи	5
1.3 Актуальность и мотивация	5
2 Обзор литературы и теоретические основы	7
2.1 RL в играх с неполной информацией: что уже сделано	7
2.2 Почему Transformer в RL — это сложно	7
2.3 Агрегация множеств: Set Transformer и PMA	8
2.4 FiLM: как подмешать глобальный контекст	8
2.5 Как бороться с комбинаторным action space	8
2.6 MCTS и планирование	9
2.7 Память и моделирование оппонента	9
2.8 Offline RL и формирование награды	9
2.9 Зачем писать собственный движок	10
3 Архитектура симулятора	11
3.1 Система сущностей (Entity System)	11
3.2 Система событий (Event System)	12
3.2.1 Типы событий	12
3.2.2 Механизм обработки событий	12
3.3 Боевая подсистема (Combat Phase)	13
3.3.1 Порядок атак	13
3.3.2 Нанесение урона	13
3.3.3 Обработка смертей	14
3.4 Ауры (stateless)	14
3.5 Таверна (экономическая подсистема)	14
3.6 Интеграция с Gymnasium	15
3.6.1 Пространство наблюдений (Observation Space)	15
3.6.2 Пространство действий (Action Space)	16
3.6.3 Функция награды (Reward Shaping)	16
4 Архитектура Transformer-агента	18
4.1 Мотивация	18
4.2 DecomposedEncoder	18
4.3 FiLM-модуляция глобальным контекстом	18
4.4 Обучаемый токен [GLOBAL_CTX]	19
4.5 GRU-шлюзы стабильности (GTrXL)	19

4.6	SwiGLU FFN и RMSNorm	20
4.7	Multi-Seed PMA	20
4.8	Symlog-трансформация	20
4.9	Конфигурация и размер модели	20
5	Пайплайн обучения	22
5.1	Алгоритм: MaskablePPO	22
5.2	Zero-Init выходных слоёв (DreamerV3)	22
5.3	Механизм Self-Play	22
5.4	Система мониторинга	22
5.5	Визуализация Attention	22
5.6	Детерминированность	23
6	Эксперименты и результаты	24
6.1	Цели экспериментов	24
6.2	Конфигурация экспериментов	24
6.3	Результаты обучения	24
6.3.1	Метрики сходимости	24
6.3.2	Метрики MLP-baseline на плато (1.5М шагов)	25
6.3.3	PvP: Transformer vs MLP	25
6.4	Анализ сходимости	26
7	Тестирование	28
8	Ограничения и направления дальнейшей работы	29
8.1	Текущие ограничения	29
8.2	Направления дальнейшей работы	29
8.2.1	Расширение Action Space	29
8.2.2	Планирование	29
8.2.3	Память и моделирование оппонента	30
8.2.4	Обучающий пайплайн	30
8.2.5	Валидация подхода: Pokémon как ближайший аналог	30
9	Заключение	32
	Список литературы	33

Аннотация

Автобаттлеры — класс стратегических игр с неполной информацией, стохастическими переходами состояний и комбинаторно обширным пространством действий, представляющий одну из открытых задач в области обучения с подкреплением (Reinforcement Learning, RL). Hearthstone Battlegrounds, насчитывающий свыше 20 миллионов активных игроков, объединяет экономическое планирование, позиционную расстановку юнитов и автоматизированную боевую фазу в единый POMDP с разреженными наградами. Несмотря на масштабность аудитории, задача обучения конкурентоспособного RL-агента для данной игры остаётся нерешённой: существующие академические работы используют упрощённые модели среды, не покрывающие ключевые механики (ауры, вложенные триггеры, Discovery).

В настоящей работе исследуется применимость Transformer-архитектуры с механизмом внимания для задач стратегического RL в автобаттлерах. Для проведения исследования был разработан headless-симулятор Hearthstone Battlegrounds с Gymnasium-интерфейсом, поддержкой Action Masking и полной реализацией боевой и экономической подсистем (свыше 3 500 строк на Python). На базе данной среды проведён сравнительный эксперимент двух архитектур: классического MLP-baseline и исследовательского Transformer-экстрактора, интегрирующего DecomposedEncoder, FiLM-модуляцию, GTrXL-шлюзы и агрегацию Pooling by Multihead Attention.

Эксперимент показал, что Transformer-агент достигает эквивалентного уровня качества игры в **5 раз быстрее** по числу шагов обучения (80 000 против 400 000 шагов до плато), подтверждая гипотезу о преимуществе структурированной обработки гетерогенных сущностей через механизм внимания. При прямом PvP-столкновении оба агента достигают устойчивого равновесия, что свидетельствует об освоении оптимальной стратегии в рамках текущей конфигурации среды и указывает на необходимость дальнейшего усложнения игрового пространства.

Ключевые слова: обучение с подкреплением, автобаттлер, Hearthstone Battlegrounds, Transformer, марковские процессы принятия решений, механизм внимания, Action Masking, Gymnasium.

1 Введение

1.1 Предметная область

Hearthstone Battlegrounds (HS:BG) — автобаттлер, являющийся одним из наиболее популярных игровых режимов коллекционной карточной игры Hearthstone, разработанной компанией Blizzard Entertainment. Режим был запущен в ноябре 2019 года и к настоящему моменту насчитывает свыше 20 миллионов активных игроков ежемесячно, что делает его одной из крупнейших площадок для исследования стратегического ИИ в карточных играх.

В отличие от классического Hearthstone, где два игрока по очереди разыгрывают карты из заранее составленных колод, в режиме Battlegrounds отсутствует этап конструирования колоды. Вместо этого восемь игроков конкурируют в серии раундов, состоящих из двух чередующихся фаз:

1. **Фаза покупок (Recruit Phase):** игрок получает золото, количество которого растёт с каждым раундом по формуле $\min(10, 3 + t - 1)$, где t — номер хода. На золото можно приобретать существ (юнитов) из таверны, продавать ненужных, повышать уровень таверны для доступа к более сильным юнитам, обновлять предложение (re-roll) и позиционировать юнитов на поле;
2. **Фаза боя (Combat Phase):** бой происходит автоматически — юниты атакуют случайных вражеских юнитов поочерёдно, пока одна из сторон полностью не уничтожена. Проигравший игрок получает урон, пропорциональный суммарному уровню (тиру) оставшихся вражеских юнитов.

Перечислим свойства предметной области, из-за которых обучение агента оказывается нетривиальным:

- **Неполная информация:** агент видит доску противника только в момент боя, руки и магазины остальных участников скрыты. Фактически задача — это POMDP, и агенту приходится восстанавливать скрытое состояние по косвенным признакам;
- **Стохастичность:** кто атакует, в кого попадает, что выпадет при roll — всё рандомизировано. Одна и та же расстановка может как победить, так и проиграть, что сильно шумит в оценке value;
- **Комбинаторный взрыв:** даже при 18 юнитах и 7 слотах на доске — число конфигураций порядка $C_{18}^7 \times 7! \approx 1.6 \times 10^8$. Баффы, attached-эффекты, рука и магазин увеличивают размерность на порядки;
- **Sparse Rewards:** итог партии определяется через 10–15 раундов, каждый из которых состоит из десятков решений (купить? продать? поставить сюда?). Установить, какое именно действие привело к победе или проигрышу, — классический credit assignment;
- **Конечный пул карт:** таверна раздаёт юнитов из общего запаса (16 копий тира 1, 15 копий тира 2 и т.д.), поэтому игроки конкурируют за одни и те же карты.

1.2 Постановка задачи

Целью настоящей работы является исследование методов обучения с подкреплением для создания автономного агента, способного вырабатывать конкурентоспособную стратегию в играх класса автобаттлеров. В качестве конкретной среды выбрана игра *Hearthstone Battlegrounds* — автобаттлер с неполной информацией, стохастической боевой фазой и комбинаторным пространством действий. Исследование носит поисковый характер: на текущем этапе рассматриваются базовые архитектуры (MLP, Transformer) и алгоритм PPO, однако проект закладывает инфраструктурную основу для апробации широкого спектра подходов — от планирования на основе MCTS до Offline RL и Opponent Modeling.

Для достижения данной цели на текущем этапе решаются следующие задачи:

1. **Разработка среды:** проектирование и реализация headless-симулятора *Hearthstone Battlegrounds* с Gymnasium-интерфейсом, детерминированным исполнением, поддержкой Action Masking и полной реализацией боевой системы с вложенными триггерами — ввиду отсутствия открытых сред, удовлетворяющих данным требованиям;
2. **Формализация MDP:** определение пространств наблюдений ($\mathbb{R}^{1009}$) и действий (Discrete(32)), функции переходов и функции награды с Reward Shaping;
3. **Исследование архитектур:** разработка и сравнение двух подходов к кодированию игровых наблюдений — классического MLP и Transformer-экстрактора со структурированной обработкой гетерогенных сущностей (DecomposedEncoder, FiLM-модуляция, GTrXL-шлюзы, PMA-агрегация);
4. **Экспериментальная валидация:** проведение сравнительных экспериментов (кривые обучения, PvP-столкновение, анализ Attention) для количественной и качественной оценки обеих архитектур.

1.3 Актуальность и мотивация

Область применения RL к карточным и стратегическим играм активно развивается. Прорывные результаты AlphaStar [19] (StarCraft II, гроссмейстерский уровень), OpenAI Five [11] (Dota 2, сверхчеловеческий уровень) и MuZero [16] (го, шахматы, сёги и Atari — сверхчеловеческий уровень) продемонстрировали потенциал RL-подходов в сложных стратегических средах. Однако каждый из этих проектов потребовал разработки специализированного симулятора, адаптированного под нужды масштабного обучения.

В контексте *Hearthstone Battlegrounds* ситуация остаётся неудовлетворительной. Проект Fireplace¹ реализует правила классического *Hearthstone*, но не поддерживает режим *Battlegrounds*. Среда на базе Unity ML-Agents требует запуска графического движка, что ограничивает скорость симуляции до ~10–50 игр в секунду (против ~1 000+ для headless-среды на Python). Коммерческие API (HSReplay, Firestone Tracker) предоставляют лишь данные наблюдений за играми реальных игроков, без возможности интерактивного управления агентом.

¹<https://github.com/jleclanche/fireplace>

Из диссертационных работ ближе всего к нашей теме стоят Leiden University [14] и LUT [24]. Обе используют упрощённые среды без аур, Discovery и позиционных эффектов. Vieira et al. [18] обучали агента для карточных игр через PPO с Action Masking — их результаты подтвердили, что наш подход реализуем. García-Sánchez et al. [4] пошли другим путём: $(\mu + \lambda)$ -ES для скоринговой функции из 21 веса, обойдясь вообще без нейросетей. Для нас это означает, что на начальном этапе можно получить бота-учителя эволюционно, а затем переключиться на Behavioral Cloning.

Хочется отдельно выделить работу Guo et al. [5]: их Transformer-агент забрался в топ-3% рейтинга Pokémon через пайплайн Offline Pre-training $\rightarrow$ Online PPO. Параллели с нашим проектом очевидны: EAV-токенизация, Summary Tokens, и — что особенно важно — отказ от Decision Transformer в пользу TD-Critic, потому что в стохастических средах кондиционирование на return не работает.

Итого: сравнение структурированных (Transformer) и плоских (MLP) подходов к кодированию гетерогенных наблюдений в автобаттлерах — задача, которая пока не имеет устоявшегося решения. Результаты нашей работы переносимы на любые среды с переменным числом сущностей и комбинаторным action space.

2 Обзор литературы и теоретические основы

2.1 RL в играх с неполной информацией: что уже сделано

Перечислим работы, из которых мы черпали идеи — как архитектурные, так и алгоритмические.

AlphaStar [19] (DeepMind, 2019) — первый агент гроссмейстерского уровня в StarCraft II. Action space там порядка 10^{26} , информация неполная, среда реального времени. DeepMind обучали его в три этапа: Behavioral Cloning на реплеях профессионалов, затем RL с Self-Play, затем League Training. Нас больше всего заинтересовали архитектурные решения: **авторегрессионный Actor** (действие генерируется по частям), **Pointer Network** [20] для указания на конкретного юнита и отдельный токен глобального контекста. Все три идеи повлияли на наш Transformer.

OpenAI Five [11] (OpenAI, 2019) показал другое: можно просто взять PPO, добавить Self-Play и дотянуть до сверхчеловеческого уровня в Dota 2. Правда, это стоило $\sim 45\,000$ лет совокупного игрового опыта и 256 GPU V100. Вывод для нас: без эффективной архитектуры sample complexity убивает проект.

MuZero [16] (DeepMind, 2020) объединил MCTS с обученной моделью мира и обыграл людей в Atari, го, шахматы и сёги. Главная идея — **Value Equivalence**: латентная модель не пытается восстановить точное состояние, а учится предсказывать value и policy напрямую. Мы планируем использовать тот же принцип для Battle Predictor — нейросети, оценивающей исход боя без полной симуляции.

MARL-GPT [1] (2024) ввёл EAV-токенизацию (Entity-Attribute-Value) для мультиагентного RL: каждая сущность кодируется набором типизированных атрибутов, а зона (доска, рука, магазин) задаётся обучаемым эмбедингом. Именно от этой работы мы отталкивались при создании **DecomposedEncoder** с Zone Embedding.

DreamerV3 [6] (Hafner et al., 2023) — World Model, работающий «из коробки» на десятках разных сред. Мы взяли оттуда две конкретные техники: (1) **symlog** для сжатия характеристик карт (АТК/HP могут расти до сотен) и (2) **Zero-Init** выходных слоёв Actor/Critic, чтобы на старте обучения не было фантомных предсказаний.

2.2 Почему Transformer в RL — это сложно

В NLP и CV Transformer давно стал стандартом, но в RL всё не так гладко. Обзор [8] выделяет три основные проблемы:

- **Non-stationarity**: политика меняется на каждом шаге обучения, а вместе с ней — распределение данных. Обычный Post-LN Transformer в таких условиях нестабилен;
- **Прожорливость к данным**: Transformer требует гораздо больше сэмплов, чем MLP. Частично это компенсируется прогревом через Behavioral Cloning или reward shaping;
- **Decision Transformer не работает в стохастике**: если кондиционировать на Return-To-Go [25], агент «заказывает» высокий return, а рандом боёв его не даёт. В нашей среде это неприемлемо.

Существует несколько архитектурных решений, которые мы используем или планируем использовать:

Pre-LN [22] — LayerNorm переносится внутрь Residual-ветки: $x = x + f(\text{Norm}(x))$. Простое изменение, но оно убирает взрыв градиентов на старте и разрешает сразу ставить большой learning rate.

GTrXL [12] — вместо $x + f(x)$ ставится GRU-шлюз. При $b_{init} = -3$ шлюз почти закрыт ($z \approx 0.05$), и Transformer на первых эпохах «молчит» — агент учится простым вещам через линейные слои. Потом градиенты сами открывают шлюз.

RMSNorm [23] — LayerNorm без вычитания среднего. Считается быстрее, а по качеству не хуже. Стандарт в LLaMA и Gemma.

SwiGLU [17] — замена ReLU-FFN на Swish-Gated Linear Unit. Чуть больше параметров (компенсируется коэффициентом $2/3$ для hidden dim), но качество заметно лучше.

Scale Anchoring [15] — auxiliary loss $\mathcal{L}_{aux} = \|s(h_t) - s_{tgt}\|^2$, который не даёт логитам «уползать в бесконечность» (logit chasing). Масштаб активаций фиксируется через замороженный ЕМА-таргет.

2.3 Агрегация множеств: Set Transformer и PMA

Доска в Battlegrounds — это множество юнитов, причём его размер меняется от хода к ходу. Для обработки таких входов мы опираемся на **Set Transformer** [7] (Lee et al., 2019). Авторы предложили два блока: **SAB** (обычный Self-Attention, $O(N^2)$) и **ISAB** (аппроксимация через M индуцированных точек, $O(NM)$). У нас $N \leq 28$ токенов (7 доска + 10 рука + 7 магазин + 3 discover + 1 глобальный), так что ISAB не окупается — используем SAB.

Для сжатия множества в фиксированный вектор взят **PMA** (Pooling by Multihead Attention): K обучаемых Seed-векторов «опрашивают» entity-токены через Cross-Attention. Каждый seed учится извлекать свой аспект ситуации — скажем, один отвечает за синергии, другой за экономику. Это значительно выразительнее наивного mean-pooling.

2.4 FiLM: как подмешать глобальный контекст

FiLM [13] (Perez et al., 2018) — аффинная модуляция: $x' = \gamma(c) \odot x + \beta(c)$, где c — вектор глобального контекста. У нас FiLM берёт глобальный стейт (золото, НР героя, тир таверны) и смещает/масштабирует эмбединги карт до Self-Attention. Это работает лучше, чем конкатенация контекста к каждому токеноу, потому что модуляция мультипликативна, а конкатенация — нет.

2.5 Как бороться с комбинаторным action space

Когда действие — это кортеж (тип, источник, цель), плоское Discrete-пространство быстро взрывается: $O(\prod |A_i|)$ комбинаций, большинство нелегальны.

SAINT [21] (Wen et al., 2025) решает это авторегрессивно: $\pi(a|s) = \prod_{i=1}^k \pi_i(a_i|s, a_{<i})$. Сеть сперва выбирает тип действия, потом карту-источник, потом цель. Выходной слой сжимается

с $O(\prod |A_i|)$ до $O(\sum |A_i|)$, а если действие не требует всех подрешений (например, Roll — без источника и цели), генерация прерывается на полпути.

Для указания на конкретную сущность из набора переменного размера нужен **Pointer Network** [20]: декодер формирует Query, берёт dot-product с Keys всех entity: $\text{logits}_i = q \cdot k_i / \sqrt{d}$. AlphaStar [19] впервые применил этот трюк для выбора юнитов в StarCraft; мы планируем использовать его при расширении action space.

2.6 MCTS и планирование

MCTS + нейросетевая Policy/Value-оценка — основа AlphaZero [16]. Формула PUCT $a_t = \arg \max_a (Q(s, a) + c \cdot P(s, a) \cdot \frac{\sqrt{N_{\text{parent}}}}{1+N_a})$ сужает поиск за счёт логитов Актора.

Проблема: классический MCTS обходит по одному листу за итерацию — GPU при этом простаивает. **RMCTS** [9] (Luo et al., 2025) обходит дерево послойно: все узлы одной глубины собираются в батч и оцениваются одним `forward()`, что даёт ускорение на порядок.

В автобаттлерах добавляется **стохастичность**: Roll таверны или исход боя — рандом, дерево взрывается. Для таких сред используют **Open-Loop MCTS**: узлы хранят не состояния, а последовательности действий; рандом пересэмплируется при каждом спуске. Совместить RMCTS с Open-Loop — одна из ближайших задач.

2.7 Память и моделирование оппонента

Battlegrounds — это POMDP: свою доску и магазин мы видим, а доску противника — только во время боя. Чтобы играть хорошо, надо запоминать прошлые наблюдения.

DTQN [3] хранит буфер из K последних наблюдений в Transformer Decoder. Attention напрямую достаёт факты из прошлого — без затухания, характерного для LSTM. Бонус: если при обучении случайно маскировать 15–20% атрибутов токеном $\langle \text{unk} \rangle$, сеть вынуждена активнее лезть в память.

Для угадывания стратегии противника интересен **Байесовский подход** [2]: по историческим данным строится матрица $P(\text{minion}_j \mid \text{minion}_i)$, и после каждого боя обновляется распределение по архетипам: $[\text{Mechs} : 0.7, \text{Beasts} : 0.1, \dots]$. Этот вектор подаётся агенту — он может подстраивать стратегию под конкретного оппонента.

2.8 Offline RL и формирование награды

Учить PPO с нуля в сложных стратегических средах дорого. Guo et al. [5] показали, что задачу можно разбить на этапы: сперва собрать траектории сильного бота, затем скопировать его поведение через Behavioral Cloning, и уже сверху дообучить Online PPO. Откуда взять бота? García-Sánchez et al. [4] продемонстрировали, что хватает простой $(\mu + \lambda)$ -эволюции скоринговой функции из 21 веса — нейросеть здесь не нужна.

Про награду. Наивный подход Decision Transformer [25] — подать на вход желаемый return и надеяться — в стохастических средах не работает: рандом боёв не даёт гарантий. Надёжнее

Reward Relabeling: переразметка состояний через TD-Critic [10], дающего стабильную $\hat{V}(s)$ вместо зашумлённого исхода. Кроме того, имеет смысл разбить награду на слагаемые $r = r_{\text{economy}} + r_{\text{synergy}} + r_{\text{combat}} + r_{\text{win}}$ [5] и постепенно отключать промежуточные — так обучение идёт быстрее, а итоговая политика не страдает.

2.9 Зачем писать собственный движок

Мы проверили все доступные открытые среды для Hearthstone Battlegrounds. Ни одна не соответствует четырём необходимым требованиям: (1) headless-исполнение на CPU, (2) Gymnasium-API, (3) Action Masking, (4) полная боевая система с цепными Deathrattle, ауры, Reborn. Поэтому пришлось писать движок с нуля.

3 Архитектура симулятора

Движок написан на Python 3.10, состоит из 11 модулей в пакете `hearthstone.engine` общим объёмом ~3 500 строк. `Gymnasium`-обёртка (`hs_env.py`) — ещё 839 строк. Движок не зависит от RL-интерфейса: его можно запускать отдельно для прогона партий, тестирования механик или генерации данных.

3.1 Система сущностей (Entity System)

Всё крутится вокруг класса `Unit` (`entities.py`, 403 строки) — обычный Python `dataclass` с фабрикой `create_from_db`. Самое важное архитектурное решение — **многоуровневые статы** (Buff Layer Stack): баффы из разных источников живут на разных слоях и не мешают друг другу (Таблица 1):

Таблица 1 – Многоуровневая система характеристик юнита (Buff Layer Stack)

Слой	Поля	Время жизни
Базовый	<code>base_atk</code> , <code>base_hp</code>	Неизменяемый, определяется <code>CARD_DB</code>
Перманентный	<code>perm_atk_add</code> , <code>perm_hp_add</code>	Навсегда: баффы от заклинаний и <code>Battlecry</code>
Ходовой	<code>turn_atk_add</code> , <code>turn_hp_add</code>	Сбрасывается в начале нового хода
Боевой	<code>combat_atk_add</code> , <code>combat_hp_add</code>	Сбрасывается после завершения боя
Аурный	<code>aura_atk_add</code> , <code>aura_hp_add</code>	Полный пересчёт при любом изменении доски

Вызов `recalc_stats()` складывает все слои и записывает итог в `cur_atk / cur_hp`. Важный нюанс: разница между потолком HP и текущим значением сохраняется. Если юнит получил урон, а затем получил бафф — максимум вырастет, но потерянные очки здоровья не восстановятся. Именно так работает оригинальная игра.

Помимо базовых статов, `Unit` хранит:

- **Флаг `is_golden`**: у золотых юнитов удвоены базовые статы, а триггеры срабатывают дважды;
- **Attached-словари** (`attached_perm`, `attached_turn`, `attached_combat`): хранят динамически навешенные `Deathrattle`-эффекты в формате `{effect_id → count}`;
- **Magnetize**: метод `magnetize_from()` «склеивает» мех-юнит с целью, перенося все баффы и `attached`-эффекты;
- **Avenge-счётчик**: для механики «Avenge (N)» — эффект стреляет после гибели N союзников.

3.2 Система событий (Event System)

Центральным модулем движка является `event_system.py` (421 строка). Здесь реализована очередь событий с приоритизированным исполнением триггеров. Без такой архитектуры невозможно правильно обработать вложенные цепочки: например, смерть юнита порождает `Deathrattle`, тот призывает нового юнита, а `Deflect-o-Bot` в ответ на призыв меха активирует `Divine Shield` — и всё это должно выполняться в детерминированном порядке.

3.2.1 Типы событий

Перечисление `EventType` определяет 17 типов, покрывающих полный жизненный цикл сущностей (Таблица 2):

Таблица 2 – Типы событий системы `EventManager`

Событие	Описание и контекст
<code>MINION_PLAYED</code>	Юнит разыгран из руки на доску (фаза покупок). Триггерит <code>Battlecry</code> .
<code>MINION_SUMMONED</code>	Юнит появился на доске (через <code>Deathrattle</code> -токен). Не триггерит <code>Battlecry</code> .
<code>MINION_DIED</code>	Юнит погиб ($HP \leq 0$). Триггерит <code>Deathrattle</code> и <code>Reborn</code> .
<code>MINION_SOLD</code>	Юнит продан через таверну. Специальные <code>sell</code> -триггеры.
<code>MINION_DAMAGED</code>	Юнит получил урон (но выжил).
<code>ATTACK_DECLARED</code>	Атака объявлена, но урон ещё не нанесён.
<code>AFTER_ATTACK</code>	Атака завершена, урон нанесён обеим сторонам.
<code>DIVINE_SHIELD_LOST</code>	Божественный щит сбит (урон поглощён).
<code>OVERKILL</code>	Нанесён урон, превышающий <code>HP</code> жертвы.
<code>START_OF_COMBAT</code>	Начало боевой фазы. Активирует <code>pre-combat</code> триггеры.
<code>START_OF_TURN</code>	Начало хода (фаза покупок).
<code>END_OF_TURN</code>	Конец хода. Сбрасывает временные эффекты.
<code>SPELL_CAST</code>	Заклинание разыграно.
<code>MINION_ADDED_TO_SHOP</code>	Юнит добавлен в магазин (для системных триггеров).

3.2.2 Механизм обработки событий

Обработка события в `EventManager` состоит из трёх стадий:

1. **Сбор** (`collect_triggers`): обходим юнитов обоих игроков. У каждого смотрим: (а) что даст `TRIGGER_REGISTRY` по его `card_id`, (б) что лежит в `attached`-словарях. Найденные триггеры

складываем в список `TriggerInstance` — там хранится ссылка на определение триггера, `UID` юнита-владельца и его позиция;

2. **Сортировка** (`order_triggers`): ключ — кортеж из пяти элементов:

(`group`, `priority`, `side`, `slot`, `uid`).

Поле `group = 0` получают «собственные» триггеры источника (`Deathrattle` умершего юнита). Благодаря этому при гибели `Scallywag` его `Deathrattle` обрабатывает раньше, чем чужой `Avenge`;

3. **Вызов**: проверяем `condition`, затем выполняем эффект $n = \text{stacks} \times (2 \text{ для } \text{golden}, 1 \text{ иначе})$ раз. Вся модификация состояния идёт через `EffectContext` — он предоставляет методы `buff_perm()`, `summon()`, `gain_gold()`, `emit_event()` и другие.

Сами события хранятся в `collections.deque`. Если триггер порождает новое событие (скажем, `Deathrattle` вызывает `MINION_SUMMONED`), оно встаёт в конец очереди и обрабатывает строго после текущего. Рекурсии нет, порядок всегда воспроизводим.

3.3 Боевая подсистема (Combat Phase)

Модуль `combat.py` (470 строк) реализует автоматическую фазу боя. Это наиболее алгоритмически сложная компонента движка, требующая корректной обработки множества пограничных случаев.

3.3.1 Порядок атак

Право первого хода определяется по размеру доски: игрок с большим числом юнитов атакует первым; при равенстве — выбор случаен. Далее атаки чередуются. Внутренний указатель атаки (`attacker_index`) циклически перемещается по доске, пропуская мёртвых юнитов и корректно обрабатывая вставки/удаления (метод `_adjust_attack_idx_after_insert`).

Поддерживаются механики:

- **Windfury**: юнит атакует дважды за ход (второй удар добавляется через повторную итерацию);
- **Immediate Attack**: токен `Scallywag` атакует сразу после призыва, без ожидания своей очереди.

3.3.2 Нанесение урона

Метод `perform_attack` реализует полный цикл атаки:

1. Генерация события `ATTACK_DECLARED`, позволяющая pre-attack триггерам модифицировать состояние;
2. Выбор защитника: с приоритетом **Taunt**-юнитов (обязательная атака); при отсутствии **Taunt** — случайный выбор;
3. Определение списка жертв с учётом **Cleave**: удар по выбранному защитнику и его соседям (до 3 юнитов одновременно);

4. Пакетное нанесение урона с обработкой **Divine Shield** (урон поглощается, щит снимается), **Poisonous/Venomous** (мгновенное убийство цели);
5. Генерация OVERKILL при избыточном уроне;
6. Взаимный удар: защитник наносит ответный урон атакующему;
7. Генерация события AFTER_ATTACK.

3.3.3 Обработка смертей

За удаление погибших юнитов отвечает `cleanup_dead`. Порядок действий для каждого трупа:

1. Снимаем `MinionSnapshot` — замороженный слепок статов и позиции. Он нужен `Deathrattle`-у: тот может, например, призвать токен «на место умершего»;
2. Собираем `Deathrattle`-триггеры через `_collect_death_triggers`. Сюда же попадает `Reborn` (копия с 1 HP);
3. Убираем юнит с доски *do* генерации события `MINION_DIED` — иначе призванный токен не найдёт свободный слот;
4. Двигаем `attacker_index`, если он указывал на удалённую позицию.

3.4 Ауры (stateless)

Модуль `auras.py` — всего 70 строк. Идея простая: при *любом* изменении доски обнуляем аурный слой у всех юнитов (`reset_aura_layer`), затем заново раздаём баффы. Три реализованные ауры:

- `Dire Wolf Alpha`: +1/ +0 соседям (золотой: +2/ +0) — позиционная;
- `Murloc Warleader`: +2/ +0 всем мурлокам (золотой: +4/ +0) — расовая;
- `Southsea Captain`: +1/ +1 всем пиратам (золотой: +2/ +2).

Такой подход («пересчитать всё с нуля») на первый взгляд расточителен, зато полностью исключает баги вроде «аура наложилась дважды при перестановке» или «аура не снялась после продажи носителя».

3.5 Таверна (экономическая подсистема)

Самый объёмный модуль — `tavern.py`, 532 строки. Здесь находится вся логика фазы покупок:

- **Золото**: каждый ход начисляется $\min(10, 3 + t - 1) + G_{next}$ монет, где G_{next} — бонус от эффектов;
- **Магазин**: n слотов (по конфигу `TAVERN_SLOTS`), карты достаются из `CardPool` с учётом тира таверны. Магазин можно заморозить;
- **Buy**: стоит 3 золота, карта уходит в руку (максимум 10 карт), из пула изымается;

- **Sell:** возвращает 1 золото. Golden-юнит при продаже превращается в 3 обычных копии, которые уходят обратно в пул. Генерируется `MINION_SOLD`;
- **Триплеты:** когда у игрока накапливается 3 не-golden копии одной карты (доска + рука), движок автоматически сливает их в golden-версию, суммируя баффы и attached-эффекты, и выдаёт заклинание `Triple Reward` — возможность взять карту тиром выше;
- **Discovery:** 3 уникальные карты из пула; выбранная изымается, остальные возвращаются;
- **Тир таверны:** повышается за золото (стоимость тикает вниз на 1 каждый ход), даёт доступ к картам высоких тиров;
- **Заклинания:** таргетированные (бафф конкретного юнита, навес `Deathrattle`) и нетаргетированные (глобальные баффы, генерация золота). Интегрированы с `event system` и `Action Masking`.

3.6 Интеграция с Gymnasium

Модуль `hs_env.py` (839 строк) реализует интерфейс `gymnasium.Env` и обеспечивает взаимодействие между RL-агентом и игровым движком. Данный модуль выполняет три ключевые функции: формирование пространства наблюдений, определение и маскирование пространства действий, а также вычисление многокомпонентной функции награды.

3.6.1 Пространство наблюдений (Observation Space)

Наблюдение формируется как плоский вектор $\mathbb{R}^{1009}$, структурированный следующим образом (Таблица 3):

Таблица 3 – Структура пространства наблюдений ($\mathbb{R}^{1009}$)

Компонент	Размер	Описание
Global Context	7	HP героя, золото, тир таверны, стоимость апгрейда, скидка на заклинания, флаг <code>Discovery</code> , флаг <code>Targeting</code>
Board	$7 \times 37 = 259$	До 7 юнитов на игровом поле
Hand	$10 \times 37 = 370$	До 10 карт в руке
Store	$7 \times 37 = 259$	До 7 позиций в магазине (включая заклинания)
Discover	$3 \times 37 = 111$	3 варианта при <code>Discovery</code>
Enemy Info	3	HP, тир и размер доски оппонента
Итого	1 009	

Каждая игровая сущность (юнит, заклинание) кодируется вектором из 37 признаков:

- **5 непрерывных:** нормализованный `card_id`, стоимость, тир, ATK, HP;

- **20 бинарных:** `is_present`, `is_spell`, `is_frozen`, 17 семантических флагов (Divine Shield, Taunt, Windfury, Poisonous, Reborn, Cleave, Stealth, Magnetic, наличие Deathrattle, Battlecry, End-of-Turn триггера, Sell-триггера, Avenge-триггера, Beast/Murloc/Pirate/Demon/Mech синергии). Семантические флаги анализируются *в реальном времени* по реестрам `TRIGGER_REGISTRY` и `SYSTEM_TRIGGER_REGISTRY`, а не хранятся в базе данных карт;
- **1 бинарный:** `is_selected` (для режима таргетирования);
- **11 категориальных:** one-hot вектор расового типа (Beast, Dragon, Demon, Murloc, Pirate, Elemental, Mech, Undead, Naga, Quilboar, Neutral).

3.6.2 Пространство действий (Action Space)

Пространство действий представлено как `Discrete(32)` с **динамическим маскированием** нелегальных действий (Таблица 4):

Таблица 4 – Пространство действий агента (`Discrete(32)`)

ID	Действие	Описание
0	End Turn	Завершить ход / отмена таргетирования
1	Roll	Обновить магазин (стоимость: 1 золото)
2–8	Buy / Target	Купить из магазина (в обычной фазе) / выбрать цель на доске (в фазе таргетирования)
9–15	Sell	Продать юнита с доски
16–25	Play Hand	Сыграть карту из руки (юнит или заклинание)
26–31	Swap Right	Переместить юнита вправо ($i \leftrightarrow i+1$)

Метод `action_masks()` возвращает булев вектор длины 32, закрывающий недоступные действия. Маска учитывает текущую фазу:

- **Обычная фаза:** Buy (при наличии золота и места), Sell (при наличии юнитов на доске), Play (при наличии карт в руке и места на доске), Roll (при наличии золота), Swap (при наличии ≥ 2 юнитов), End Turn;
- **Discovery:** разрешены только действия 2–4 (выбор одной из 3 опций);
- **Targeting:** разрешены действия 2–8 (выбор цели на доске) и 0 (отмена). Для механики Magnetize маска дополнительно фильтрует только юнитов типа MECH.

3.6.3 Функция награды (Reward Shaping)

Функция награды разработана как **многокомпонентная** для ускорения сходимости в пространстве с разреженными наградами:

$$r_{total} = r_{combat} + r_{economy} + r_{board} + r_{terminal} + r_{synergy}, \quad (1)$$

- $r_{combat} = 0.2 \cdot (dm_{gdealt} - dm_{gtaken})$ — разница урона после автоматического боя. Поощряет агента строить сильные доски, наносящие максимальный урон оппоненту;

- $r_{economy} = -0.1 \cdot gold_{unspent}$ — штраф за неиспользованное золото при нажатии End Turn. Мотивирует эффективное расходование ресурсов;
- $r_{board} = 0.05 \cdot \Delta(board_power)$ — бонус за прирост совокупной силы доски. Метрика `board_power` вычисляется как $\sum_{u \in board} \sqrt{u_score}$, где u_score учитывает АТК, НР и мультипликаторы за ключевые слова: Divine Shield ($\times 1.5$), Poisonous ($\times 2.0$), Windfury ($\times 1.5$), Reborn ($\times 1.3$), Cleave ($\times 1.4$);
- $r_{terminal} = +5.0$ за победу, -5.0 за поражение;
- $r_{synergy}$: $+2.5$ за триплет (слияние 3 копий), $+0.5$ за пару (2 копии), $+3.0$ за розыгрыш Triple Reward (Discovery тиром выше).

4 Архитектура Transformer-агента

Центральным исследовательским вкладом настоящей работы является разработка Transformer-архитектуры (модуль `trans.py`, 435 строк), интегрирующей передовые техники из области RL и NLP для структурированной обработки гетерогенных игровых наблюдений. Архитектура реализована в виде SB3-совместимого `BaseFeaturesExtractor` и может использоваться как drop-in замена стандартного MLP-экстрактора.

4.1 Мотивация

Наивный подход — подача плоского вектора $\mathbb{R}^{1009}$ в MLP — имеет фундаментальные ограничения:

1. MLP не различает семантические границы между зонами (доска, рука, магазин);
2. Полносвязные слои не улавливают **внутри-множественные зависимости** между юнитами (расовые синергии, взаимодействие Deathrattle + Baron Rivendare);
3. Фиксированная входная размерность не позволяет обрабатывать переменное число непустых слотов — MLP тратит параметры на обработку нулевых padding-токенов.

Transformer-архитектура решает эти проблемы через: (a) токенизацию каждого юнита как отдельного вектора, (b) Self-Attention для обнаружения межзонных зависимостей, (c) padding mask для игнорирования пустых слотов.

4.2 DecomposedEncoder

Вместо наивного `Linear(37, d_model)`, входной вектор сущности разделяется на три семантические группы, обрабатываемые независимыми проекциями:

1. **Continuous** (5 признаков): `card_id_norm`, `cost`, `tier`, `ATK`, `HP` — symlog-совместимые скаляры $\rightarrow \text{Linear}(5, d_model)$;
2. **Binary** (20 признаков): `is_present`, `is_spell`, `is_frozen`, 17 флагов ключевых слов/эффектов $\rightarrow \text{Linear}(20, d_model)$;
3. **Types** (11 признаков): one-hot расовый тип $\rightarrow \text{Linear}(11, d_model)$.

Результаты суммируются (аддитивное смешивание, аналогично `token + position embedding` в BERT [22]) и дополняются **Zone Embedding** — обучаемым вектором, идентифицирующим принадлежность слота к зоне (`Board` = 1, `Hand` = 2, `Store` = 3, `Discover` = 4, пустой = 0). Данный подход мотивирован архитектурой MARL-GPT [1], использующей аналогичный механизм для мультиагентных сред.

4.3 FiLM-модуляция глобальным контекстом

Перед входом в Transformer-блоки, токены сущностей модулируются глобальным контекстом игры через механизм FiLM [13]:

$$x'_i = (\gamma(c) + 1) \odot x_i + \beta(c), \quad (2)$$

где $c \in \mathbb{R}^{10}$ — конкатенация глобального вектора (7: HP, золото, тир, стоимость апгрейда, скидка, Discovery, Targeting) и информации о противнике (3: HP, тир, размер доски), а $\gamma, \beta = \text{MLP}(c)$ — параметры модуляции.

Критически важная деталь реализации — **Zero-Init**: веса последнего слоя MLP-генератора инициализируются нулями. В сочетании с формулой $(\gamma + 1)$ это гарантирует, что на шаге 0 FiLM работает как тождественная функция ($x' = x$), не нарушая начальное обучение [6].

4.4 Обучаемый токен [GLOBAL_CTX]

FiLM модулирует токены *do* Self-Attention (pre-Attention conditioning). Однако карты при этом не могут *обратиться* к макро-контексту *внутри* Attention. Для решения этой проблемы добавлен обучаемый токен [GLOBAL_CTX] — аналог [CLS] из BERT и scalar features entity из AlphaStar [19]:

$$\text{ctx} = \text{MLP}([\text{global}(7) \parallel \text{enemy}(3)]) \in \mathbb{R}^{d_{\text{model}}}.$$

Этот токен конкатенируется к множеству entity-токенов и участвует в Self-Attention наравне с картами, позволяя механизму внимания решать: “у нас 2 ХП — берём статьи, а не экономику”. FiLM и [GLOBAL_CTX] не взаимоисключающие: FiLM = pre-Attention модуляция (“настроить”), [GLOBAL_CTX] = in-Attention консультация (“спросить”).

4.5 GRU-шлюзы стабильности (GTrXL)

Каждый Transformer-блок использует **GRU-подобные шлюзы** вместо стандартного Residual-сложения [12]:

$$z = \sigma(W_z \cdot [x, y] + b_z), \quad (3)$$

$$r = \sigma(W_r \cdot [x, y] + b_r), \quad (4)$$

$$\hat{h} = \tanh(W_h \cdot [r \odot x, y]), \quad (5)$$

$$\text{out} = (1 - z) \odot x + z \odot \hat{h}, \quad (6)$$

Здесь x — вход (Residual), y — результат Attention или FFN. Ключ к стабильности — инициализация $b_z = -3$: при старте $z = \sigma(-3) \approx 0.05$, и Transformer по сути пропускает вход без изменений ($\text{out} \approx x$). Агент на первых эпохах учится простым вещам — покупать юнитов, тратить золото — используя только DecomposedEncoder. Затем градиенты постепенно «открывают шлюз», и Transformer подключается к поиску сложных зависимостей между юнитами.

Шлюзы стоят на обоих sub-layers — и на Attention, и на FFN.

4.6 SwiGLU FFN и RMSNorm

FFN-блок заменён на **SwiGLU** [17]. Скрытая размерность берётся как $2/3$ от стандартной и округляется до кратного 256 (так быстрее на GPU):

$$\text{SwiGLU}(x) = W_2 \cdot (\text{SiLU}(W_1 x) \odot Vx). \quad (7)$$

Для нормализации взят **RMSNorm** [23] вместо LayerNorm. Он масштабирует по среднему квадрату без вычитания среднего: $\hat{x}_i = x_i \cdot g_i / \text{RMS}(x)$. По точности разницы с LayerNorm почти нет, зато вычислительно дешевле — RMSNorm сейчас стандарт в LLaMA, Gemma, Mistral.

4.7 Multi-Seed PMA

Для агрегации множества entity-токенов в единое представление фиксированной размерности используется **Pooling by Multihead Attention** (PMA) [7] с $K = 4$ обучаемыми Seed-векторами. Каждый seed выступает как Query в Cross-Attention к entity-токенам:

$$\text{PMA}(X) = \text{rFF}(\text{MultiHead}(S, X, X)),$$

где $S \in \mathbb{R}^{K \times d_{\text{model}}}$ — обучаемые Seed-ы. Четыре seed-а извлекают различные аспекты оценки доски: синергии, экономическая ценность, оборонительный потенциал, наступательная сила. Результаты конкатенируются и проецируются в $\mathbb{R}^{d_{\text{model}}}$ через линейный слой.

PMA заменяет наивный mean-pooling, который уничтожает найденные в Self-Attention синергии, усредняя информацию по всем юнитам.

4.8 Symlog-трансформация

Перед подачей в DecomposedEncoder, все скалярные значения сущностей (ATK, HP, tier, cost) обрабатываются симметричным логарифмом из DreamerV3 [6]:

$$\text{symlog}(x) = \text{sign}(x) \cdot \ln(|x| + 1). \quad (8)$$

Это безопасно сжимает потенциально экспоненциально растущие статые (в поздней игре ATK/HP могут достигать сотен) в узкий диапазон, сохраняя высокую точность около нуля.

4.9 Конфигурация и размер модели

Итоговая конфигурация Transformer-экстрактора приведена в Таблице 5.

Таблица 5 – Конфигурация Transformer-экстрактора

Параметр	Значение
d_{model}	128
Число голов внимания	4

Число Transformer-блоков	4
FFN	SwiGLU (hidden = 256, nearest multiple of 256)
Нормализация	RMSNorm (Pre-LN)
Шлюзы	GTrXL GRU ($b_{init} = -3$) на Attention и FFN
Агрегация	Multi-Seed PMA ($K = 4$)
FiLM context	10 (global 7 + enemy 3)
Symlog	Включён
Параметры экстрактора	~600 000
Параметры всего агента	~750 000

5 Пайплайн обучения

5.1 Алгоритм: MaskablePPO

Мы используем **MaskablePPO** из `sb3-contrib` [18]. Его главная особенность — встроенная поддержка Action Masking. Перед Softmax логитам нелегальных действий выставляется $-\infty$, поэтому агент физически не может выбрать невозможный ход. Для нашей среды, где набор допустимых действий меняется каждый тик, это обязательное требование.

5.2 Zero-Init выходных слоёв (DreamerV3)

Сразу после создания модели мы зануляем веса и bias последних слоёв Actor (`action_net`) и Critic (`value_net`), следуя рецепту DreamerV3 [6]. В итоге на нулевой эпохе логиты равны нулю (равномерная политика), а Value тоже ноль — никаких «фантомных» предсказаний, чистый старт.

5.3 Механизм Self-Play

Для обеспечения нетривиального оппонента используется механизм **Self-Play** [19], реализованный в `SelfPlayCallback`:

1. Каждые 50 000 шагов текущая модель сохраняется и загружается как оппонент на CPU;
2. Во всех параллельных средах обновляется метод `set_opponent()`;
3. Оппонент использует `model.predict()` с `deterministic=True` для принятия решений.

Это позволяет агенту обучаться против всё более сильного противника, предотвращая переобучение к фиксированной стратегии.

5.4 Система мониторинга

Обучение контролируется тремя специализированными Callback-ами:

- **BoardPowerCallback**: каждые 2 000 шагов логирует среднюю и максимальную “силу доски” — кастомную метрику качества построения, учитывающую ATK, HP и ключевые слова юнитов;
- **GameLoggerCallback**: каждые 15 000 шагов прогоняет полную партию в отдельной среде и сохраняет пошаговый лог в Markdown-формате, позволяя визуально анализировать стратегию агента;
- **WandbCallback**: логирование всех метрик и градиентов в Weights & Biases.

5.5 Визуализация Attention

Разработан специализированный скрипт `visualize_attention.py` (227 строк) для анализа паттернов внимания обученного Transformer-агента. Скрипт регистрирует forward-hooks на все `MultiHeadAttention`-слои, перехватывает attention weights и строит:

- Средневзвешенные тепловые карты для каждого слоя (averaged over heads);
- Детальные карты каждой отдельной головы первого слоя.

Оси хитмапов размечены метками зон: [CTX] для глобального токена, B0–B6 для доски, H0–H9 для руки, S0–S6 для магазина, D0–D2 для Discovery. Пустые слоты помечаются скобками.

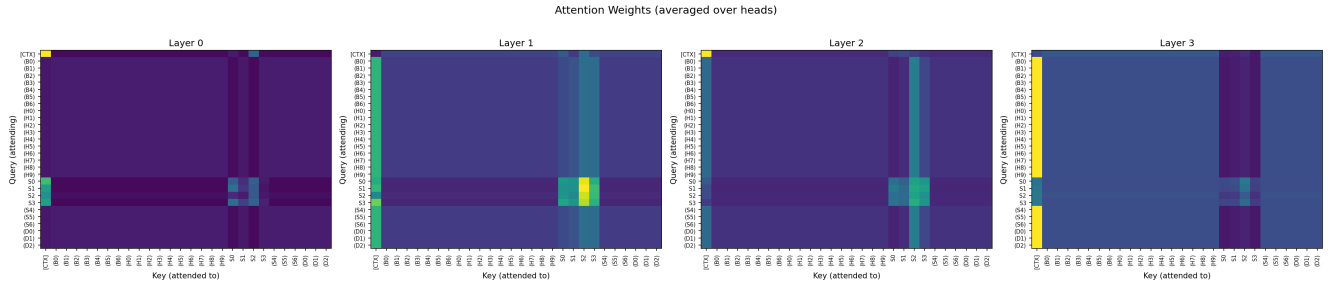


Рис. 1: Средневзвешенные карты внимания по слоям Transformer-агента. На Layer 0 внимание концентрируется на юнитах доски (B0–B6) и токене контекста [CTX]; на Layer 3 формируются устойчивые паттерны кросс-зонного взаимодействия между доской и магазином

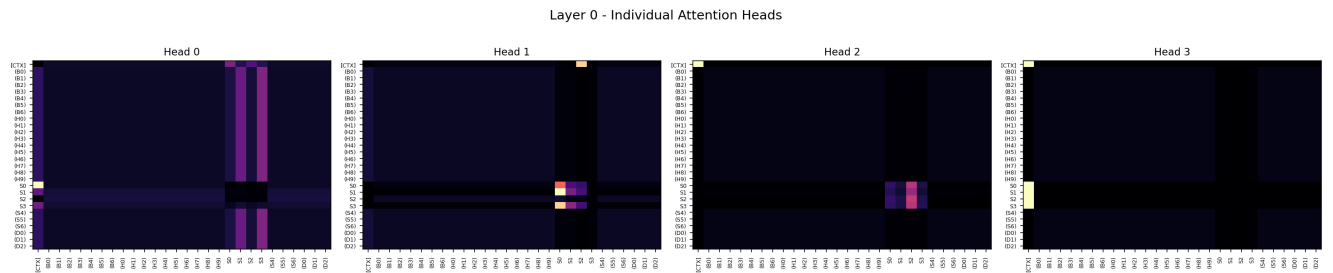


Рис. 2: Карты внимания отдельных голов Layer 0. Head 0 специализируется на внутридосковых связях; Head 1 фокусируется на слотах магазина; Head 3 — на глобальном контексте и Discovery-зоне

5.6 Детерминированность

Для научной воспроизводимости экспериментов реализована функция `setup_determinism(seed)`, устанавливающая:

- `PYTHONHASHSEED` для детерминированного хеширования;
- `CUBLAS_WORKSPACE_CONFIG` для детерминированных CUDA-операций;
- `torch.backends.cudnn.deterministic = True`;
- `set_random_seed()` из SB3 для NumPy, PyTorch и Python random.

6 Эксперименты и результаты

6.1 Цели экспериментов

Экспериментальная часть преследует три цели:

1. **Валидация среды:** подтверждение корректности формирования наблюдений, масок, наград и прохождения градиентов;
2. **Демонстрация сходимости:** обученный агент должен демонстрировать осмысленное поведение (покупка юнитов, заполнение доски, использование триплетов);
3. **Сравнение архитектур:** оценка преимуществ Transformer-экстрактора по сравнению с MLP-baseline.

6.2 Конфигурация экспериментов

Основные параметры обучения для обеих архитектур сведены в Таблице 6.

Таблица 6 – Конфигурация экспериментов

Параметр	MLP Baseline	Transformer
Алгоритм	MaskablePPO	MaskablePPO
Архитектура	MLP [512, 512]	Transformer (4L, 4H, 128D)
Learning Rate	3×10^{-4}	3×10^{-4}
γ (discount)	0.99	0.99
Batch Size	256	256
n_steps	2 048	2 048
Entropy Coefficient	0.01	0.01
Параллельные среды	4	4
Всего шагов	1 500 000	300 000
GPU	T4 (Kaggle)	T4 (Kaggle)
Self-Play	50 000 шагов	50 000 шагов
Zero-Init	Нет	Да (Actor + Critic)

6.3 Результаты обучения

6.3.1 Метрики сходимости

Ключевым экспериментальным результатом является **скорость сходимости** (sample efficiency). Результаты сведены в Таблице 7. Обе архитектуры были обучены до достижения плато по метрике `avg_board_power` ≈ 15 :

Таблица 7 – Сравнение скорости сходимости (Sample Efficiency)

Метрика	MLP	Transformer
Шаги до плато (~ 15 board power)	$\sim 400\,000$	$\sim 80\,000$
Ускорение (sample efficiency)	$1\times$	$\sim 5\times$
Inference speed	~ 600 fps	~ 200 fps

Transformer сходится в ~ 5 раз быстрее по количеству шагов обучения. При этом каждый шаг Transformer-итерации дороже вычислительно ($\sim 3\times$ медленнее per iteration из-за $O(N^2)$ Self-Attention), но суммарная выгода по числу необходимых взаимодействий со средой (Net Sample Efficiency) остаётся в пользу Transformer.

6.3.2 Метрики MLP-baseline на плато (1.5M шагов)

Таблица 8 – Метрики MLP-baseline на плато (1.5M шагов)

Метрика	Значение	Интерпретация
Explained Variance	~ 0.7	Value function корректно предсказывает return
Value Loss	~ 3.0	Стабильная оценка value с высокой дисперсией
Policy Gradient Loss	~ -0.03	Стабильная оптимизация политики
Entropy Loss	~ -0.85	Политика уверена в решениях
Avg Board Power	~ 15	Потолок текущей конфигурации среды

`explained_variance` ≈ 0.7 говорит о том, что Critic неплохо предсказывает return — вся цепочка «среда $\rightarrow$ obs $\rightarrow$ нейросеть $\rightarrow$ gradient» работает. По поведению оба агента играют осмысленно: покупают юнитов, заполняют доску до 7 слотов, собирают триплеты и через Triple Reward берут карты высоких тиров.

Замечание: Transformer обучался 300K шагов (лимит GPU Kaggle T4), MLP — 1.5M. Тем не менее Transformer вышел на плато за $\sim 5\times$ меньше сэмплов.

6.3.3 PvP: Transformer vs MLP

Мы сравнили обученных агентов (MLP после 1.5M шагов, Transformer после 300K) через `evaluate_pvp.py` (Таблица 9):

Таблица 9 – Результаты прямого столкновения Transformer vs MLP

Игра	HP Transformer	HP MLP	Δ HP
1	22	21	+1
2	30	25	+5
3	24	27	-3

Главный результат PvP: **все три матча ушли в таймаут** (>500 ходов). Ни один агент не смог убить другого — HP колебалось в районе 20–30 на протяжении всей партии.

Что из этого следует:

1. **Обе архитектуры «решили» задачу:** и MLP, и Transformer научились строить максимально живучие доски. При 18 юнитах и тирах 1–3 пространство стратегий не настолько велико, чтобы разница в архитектуре дала преимущество в PvP;
2. **Среда слишком простая:** то, что MLP и Transformer играют одинаково сильно, говорит скорее о потолке сложности нашей среды. Чтобы Transformer мог показать себя, нужно больше карт и кросс-расовые синергии;
3. **Среда реализована корректно:** сам факт стабильного равновесия — хороший знак. Агенты не нашли эксплойтов, не зациклились, а выработали осмысленную стратегию — значит, движок работает правильно.

6.4 Анализ сходимости

Ни MLP, ни Transformer не показали катастрофического забывания или коллапса политики — обучение шло стабильно на всём протяжении. Мы выделили три характерные фазы (границы даны для MLP; Transformer проходит те же этапы примерно в 5 раз быстрее):

1. **«Научиться тратить золото»** (MLP: 0–200K, Transformer: 0–40K): агент понимает, что надо покупать юнитов, когда есть деньги, и жать End Turn, когда денег нет. Board Power растёт с 0 до ~ 8 ;
2. **«Синергии и триплеты»** (MLP: 200K–800K, Transformer: 40K–160K): появляются расовые связки, сбор триплетов, использование Discovery. Board Power доходит до ~ 13 ;
3. **«Финальная шлифовка»** (MLP: 800K–1.5M, Transformer: 160K–300K): агент учится тонкостям — когда повышать тир таверны, как расставлять юнитов. Board Power выходит на плато ~ 15 –18.

Динамика обучения наглядно представлена на Рисунке 3: кривые loss-функций демонстрируют устойчивое снижение, при этом Transformer (розовая линия) сходится существенно быстрее MLP-baseline (оранжевая линия).

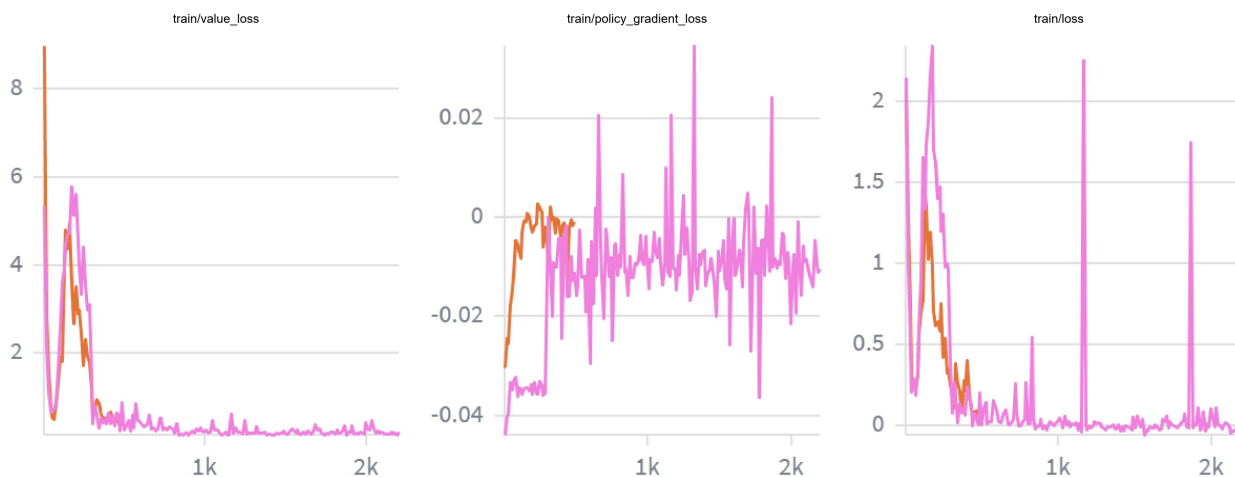


Рис. 3: Динамика ключевых метрик обучения (WandB): value_loss, policy_gradient_loss и train_loss для MLP (оранжевая) и Transformer (розовая). Transformer достигает стабильных значений loss в ~ 5 раз быстрее по числу шагов

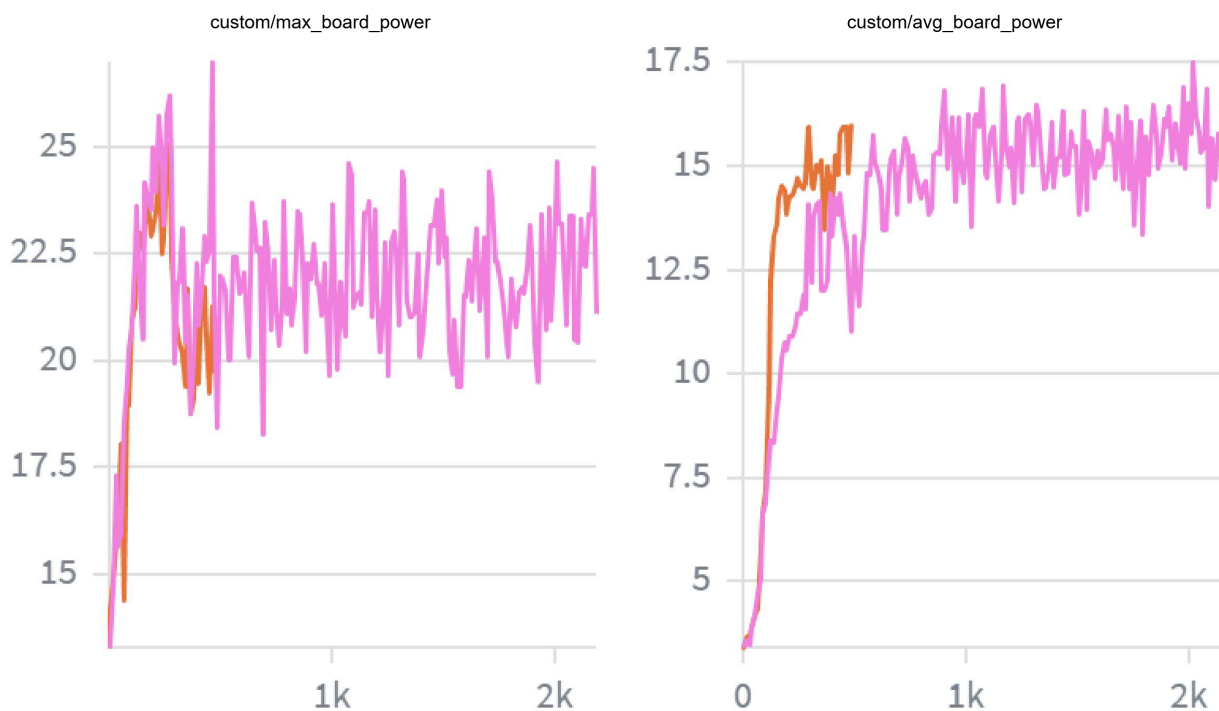


Рис. 4: Динамика board_power (WandB): максимальная и средняя сила доски агентов. Transformer (розовая) достигает плато (~ 15) на порядок быстрее MLP (оранжевая), подтверждая преимущество в sample efficiency

Помимо фазовой структуры, наблюдался **эффект Self-Play**: после обновления оппонента (каждые 50 000 шагов) фиксировались кратковременные просадки value loss — агент адаптируется к новому стилю игры противника, — после чего метрики стабилизировались на более высоком уровне.

GTrXL-шлюзы и Zero-Init FiLM были включены в архитектуру на основе теоретических рекомендаций из [12] и [6] соответственно. Полноценный ablation study (отключение отдельных компонентов) не проводился из-за ограничений вычислительных ресурсов и является предметом дальнейшей работы.

7 Тестирование

Корректность реализации движка подтверждена обширным набором из **13 тестовых модулей** общим объёмом нескольких тысяч строк. Тесты организованы по принципу “белого ящика” и покрывают:

- **Модульные тесты сущностей** (`test_entities.py`): создание юнитов из базы данных, многоуровневая система баффов (перманентные, ходовые, боевые, аурные), корректность Golden-версий с удвоенными стататами, механика Magnetize (объединение Mech-юнитов с суммированием баффов и attached-эффектов), метод `restore_stats()` с сохранением потери HP;
- **Тесты системы событий** (`test_event_system.py`): корректность порядка триггеров (`group` → `priority` → `side` → `slot` → `uid`), вложенные цепочки (Deathrattle → SUMMONED → Deflect-o-Bot trigger), обработка Golden-юнитов (двойное срабатывание), корректность EffectContext (`buff_perm`, `summon`, `gain_gold`);
- **Тесты боевой системы** (`test_combat.py`): Deathrattle + Reborn (призыв копии с 1 HP после смерти), Cleave (урон по 3 юнитам одновременно), Divine Shield (поглощение первого удара), Poisonous/Venomous (мгновенное убийство), Windfury (двойная атака), Immediate Attack (внеочередная атака токена), Taunt-приоритет, корректность вычисления урона герою;
- **Тесты таверны** (`test_tavern.py`): покупка (проверка золота, места в руке), продажа (возврат золота и карт в пул), триплеты (слияние 3 копий с суммированием баффов, выдача Triple Reward), Discovery (выбор из 3 уникальных карт, изъятие из пула, возврат невыбранных), заморозка магазина, апгрейд таверны, розыгрыш заклинаний (таргетированных и нетаргетированных);
- **Тесты пула карт** (`test_pool.py`): конечность запаса (изъятие и возврат копий), вероятностная выборка по тирам, Discovery с unique constraint;
- **Интеграционные тесты** (`test_integration.py`): полные партии от начала до определения победителя, корректность Self-Play (два агента с разными стратегиями);
- **Тесты RL-среды** (`test_rl_env.py`): корректность Action Masking (все невозможные действия замаскированы), формирование наблюдений (размерность 1009, нормализация значений), корректность reset/step циклов;
- **Тесты Golden-логики** (`test_golden_logic.py`): полный цикл триплетов для каждого юнита, корректность golden Battlecry (например, Golden Alleycat призывает Golden Tabbycat);
- **Тесты новых механик** (`test_new_mechanics.py`): Kaboom Bot (4 урона случайному врагу), Deflect-o-Bot (DS при призыве мехов), Spawn of N’Zoth (+1/+1 всем при смерти);
- **Тесты покрытия пробелов** (`test_coverage_gaps.py`): пограничные случаи — пустая доска при продаже, переполнение руки, Discovery при пустом пуле.

8 Ограничения и направления дальнейшей работы

8.1 Текущие ограничения

1. **Ограниченный пул карт:** текущая реализация покрывает подмножество юнитов и заклинаний первых тиров; в оригинальной игре представлено свыше 200 уникальных карт тиров 1–6. Расширение пула является приоритетной задачей, при этом архитектура движка *не требует модификации* — достаточно добавить записи в CARD_DB и триггеры в TRIGGER_REGISTRY;
2. **Формат 1v1:** симулируется дуэль двух игроков, тогда как оригинал предполагает 8 участников с ротацией пар. Расширение до 8 игроков потребует модификации класса Game и среды, но не затронет ядро движка;
3. **Вычислительные ресурсы:** обучение проводилось на NVIDIA Tesla T4 (16 GB VRAM, Kaggle), что ограничило длительность экспериментов и не позволило провести полноценный ablation study;
4. **Отсутствие героев:** в оригинальной игре каждый игрок выбирает героя с уникальной Hero Power, создающей дополнительный слой стратегического разнообразия;
5. **Эволюция метагейма:** Hearthstone Battlegrounds регулярно обновляется разработчиком — добавляются новые юниты, расы, механики и заклинания, что непрерывно увеличивает комбинаторную сложность среды. Поддержание актуальности симулятора потребует систематического обновления базы данных карт и реестра триггеров.

8.2 Направления дальнейшей работы

Проект задумывался как исследовательская платформа, а не как конечный продукт. Ниже — конкретные направления, в которые мы планируем двигаться.

8.2.1 Расширение Action Space

Сейчас действие — один элемент из Discrete(32). При расширении пула карт это перестанет работать. Решение — **авторегрессионная декомпозиция** по схеме SAINT [21]: агент сначала выбирает тип действия, затем источник через Pointer Network [20], затем цель. Это снижает сложность с $O(\prod |A_i|)$ до $O(\sum |A_i|)$, плюс Roll и End Turn можно завершать после первого шага, не генерируя лишних действий.

8.2.2 Планирование

Стоит попробовать **Breadth-First RMCTS** [9] — GPU-батчиrowанный MCTS, заточенный под стохастические среды. Его ключевые идеи: pre-allocated деревья в contiguous memory и prior policy для распределения бюджета. Авторы показывают $\sim 10\times$ ускорение по сравнению с UCB-MCTS при том же бюджете.

Вторая идея — **Latent Battle Predictor** в духе MuZero [16]. Вместо того чтобы симулировать бой Monte-Carlo, можно обучить сеть предсказывать исход за $O(1)$ по эмбедингам сущностей.

Оптимизация по Value Equivalence [10]: сеть не воспроизводит пошаговый бой, а сразу даёт скалярную оценку.

8.2.3 Память и моделирование оппонента

Моделирование противника [2]: идея — байесовский классификатор, который по наблюдениям доски противника обновляет вероятности архетипов («мурлоки», «мехи» и т.д.). Агент сможет адаптировать покупки, не храня в памяти все прошлые ходы.

DTQN [3]: Transformer Decoder, который хранит буфер из K последних наблюдений. Attention напрямую достаёт нужные факты из прошлого — без затухания, характерного для LSTM/GRU. Авторы также предлагают аугментацию $\langle \text{unk} \rangle$ -токенами, чтобы сеть не «ленилась» и действительно использовала память.

8.2.4 Обучающий пайплайн

Итеративный пайплайн BC $\rightarrow$ PPO $\rightarrow$ Self-Play [5]: вместо чистого PPO — многоэтапная схема. Сначала коэволюционно выращиваем бота-учителя [4], затем Behavioral Cloning для прогрева весов, потом Online PPO для выхода за пределы эвристики, и наконец Self-Play для генерации нового датасета. Этапы PPO + Self-Play повторяются итеративно.

Категориальный Critic (Two-Hot) [6]: вместо скалярного Value Head — распределение из 255 бинов в symlog-пространстве. Это устойчивее к выбросам и позволяет работать с мультимодальными return-ами. Вдобавок Percentile Return Normalization: деление на $S = \max(\text{Perc}_{95} - \text{Perc}_5, 1)$ вместо стандартного μ/σ .

Decomposed Reward Shaping [5]: текущую функцию награды имеет смысл разложить на компоненты (r_{economy} , r_{synergy} , r_{win}) и постепенно убирать шейпинговые слагаемые — иначе агент рано или поздно начнёт эксплуатировать промежуточные метрики вместо того, чтобы побеждать.

8.2.5 Валидация подхода: Pokémon как ближайший аналог

Ключевым ориентиром для дальнейшего развития проекта является работа Guo et al. [5], в которой Transformer-агент достиг уровня топ-3% рейтинговых игроков в конкурентных боях Pokémon. Эта задача структурно близка к Hearthstone Battlegrounds:

- **Общие черты:** гетерогенные сущности с десятками числовых и категориальных атрибутов, стохастический исход боя (критические удары, промахи $\leftrightarrow$ рандом ударов), необходимость долгосрочного планирования (выбор команды $\leftrightarrow$ построение доски), огромное пространство карт/покемонов;
- **Покемоны проще:** пространство действий $|\mathcal{A}| \approx 9$ (4 атаки + switch), полная наблюдаемость своей команды, отсутствие фаз покупки/продажи и экономической подсистемы;
- **BG сложнее:** комбинаторное пространство действий (покупка $\times$ позиция $\times$ продажа $\times$ заклинание), неполное знание магазина (Roll как стохастический запрос), экономическое планирование (баланс gold $\leftrightarrow$ tier-up $\leftrightarrow$ freeze), фаза авто-боя без контроля.

Несмотря на существенно большую сложность BG, архитектурный пайплайн Pokémon-агента практически полностью переносим на нашу задачу: EAV-токенизация сущностей, итеративный BC $\rightarrow$ PPO $\rightarrow$ Self-Play, Decomposed Rewards с затуханием, отказ от Decision Transformer в пользу TD-Critic из-за дисперсии боёв, и Summary Tokens для мультимодального входа. Успех данного подхода на более простой, но структурно аналогичной задаче является сильным аргументом в пользу предложенной исследовательской программы.

9 Заключение

Настоящая работа представляет первый этап исследовательского проекта по применению методов обучения с подкреплением к жанру автобаттлеров на примере Hearthstone Battlegrounds. Основные результаты:

1. **Исследовательская платформа:** спроектирован и реализован модульный headless-симулятор на Python с системой событий, боевой подсистемой с поддержкой 10 ключевых механик и экономической подсистемой (покупка, продажа, триплеты, Discovery, заклинания). Симулятор интегрирован с Gymnasium ($\mathbb{R}^{1009}$, $|\mathcal{A}| = 32$, Action Masking) и покрыт 13 тестовыми модулями;
2. **Transformer-архитектура:** спроектирован Features Extractor, интегрирующий DecomposedEncoder, FiLM-модуляцию с Zero-Init, GTrXL-шлюзы ($z_0 \approx 0.05$), Multi-Seed PMA ($K = 4$), SwiGLU FFN, RMSNorm и symlog-трансформацию;
3. **Экспериментальная валидация:** Transformer-агент достигает эквивалентного уровня качества игры в **~5 раз быстрее** по числу шагов обучения (80K vs 400K шагов до плато). При прямом столкновении оба агента достигают устойчивого равновесия, что свидетельствует об освоении оптимальной стратегии в рамках текущей конфигурации среды.

Полученные результаты подтверждают жизнеспособность Transformer-архитектуры для структурированных наблюдений в RL-средах с гетерогенными сущностями. Разработанная платформа спроектирована с учётом масштабируемости и не требует модификации ядра при расширении пула карт и механик. Дальнейшие направления работы — авторегрессионная декомпозиция действий (SAINT), GPU-батчирующий RMCTS, многоэтапный пайплайн BC $\rightarrow$ PPO $\rightarrow$ Self-Play и моделирование оппонента — определяют переход от текущего PPO-baseline к агенту, способному конкурировать с опытными игроками.

Список литературы

Список литературы

- [1] Anonymous. MARL-GPT: Multi-Agent Reinforcement Learning with GPT // OpenReview Submission. — 2024. — Режим доступа: <https://anonymous.4open.science/r/marl-gpt-20365>, свободный (дата обращения: 30.03.2026).
- [2] Bursztein, E. Predicting Hearthstone Opponent Deck Using Machine Learning // elie.net blog. — 2018. — Режим доступа: <https://elie.net/blog/hearthstone/>, свободный (дата обращения: 30.03.2026).
- [3] Esslinger, K., Platt, R., Amato, C. Deep Transformer Q-Networks for Partially Observable Reinforcement Learning // arXiv preprint. — 2022. — arXiv:2206.01078. — Режим доступа: <https://arxiv.org/abs/2206.01078>, свободный (дата обращения: 30.03.2026).
- [4] García-Sánchez, P. et al. Evolutionary Optimization of Hearthstone Agents // arXiv preprint. — 2024. — arXiv:2410.19681. — Режим доступа: <https://arxiv.org/abs/2410.19681>, свободный (дата обращения: 30.03.2026).
- [5] Guo, Y. et al. Pokémon Agent: Reinforcement Learning for Turn-Based Strategy Games // arXiv preprint. — 2025. — arXiv:2504.04395. — Режим доступа: <https://arxiv.org/abs/2504.04395>, свободный (дата обращения: 30.03.2026).
- [6] Hafner, D., Pasukonis, J., Ba, J., Lillicrap, T. Mastering Diverse Domains through World Models // arXiv preprint. — 2023. — arXiv:2301.04104. — Режим доступа: <https://arxiv.org/abs/2301.04104>, свободный (дата обращения: 30.03.2026).
- [7] Lee, J., Lee, Y., Kim, J. et al. Set Transformer: A Framework for Attention-based Permutation-Invariant Input // Proceedings of ICML. — 2019. — arXiv:1810.00825. — Режим доступа: <https://arxiv.org/abs/1810.00825>, свободный (дата обращения: 30.03.2026).
- [8] Li, W. et al. A Survey on Transformers in Reinforcement Learning // arXiv preprint. — 2023. — arXiv:2307.05979. — Режим доступа: <https://arxiv.org/abs/2307.05979>, свободный (дата обращения: 30.03.2026).
- [9] Luo, S. et al. Breadth-First RMCTS: GPU-Batched Monte-Carlo Tree Search // arXiv preprint. — 2025. — arXiv:2601.01301. — Режим доступа: <https://arxiv.org/abs/2601.01301>, свободный (дата обращения: 30.03.2026).
- [10] Oh, J. et al. Discovering Object-Centric Generalized Value Functions From Pixels // arXiv preprint. — 2022. — arXiv:2209.00588. — Режим доступа: <https://arxiv.org/abs/2209.00588>, свободный (дата обращения: 30.03.2026).

- [11] OpenAI. Dota 2 with Large Scale Deep Reinforcement Learning // arXiv preprint. — 2019. — arXiv:1912.06680. — Режим доступа: <https://arxiv.org/abs/1912.06680>, свободный (дата обращения: 30.03.2026).
- [12] Parisotto, E., Song, H. F., Rae, J. W. et al. Stabilizing Transformers for Reinforcement Learning // Proceedings of ICML. — 2020. — arXiv:1910.06764. — Режим доступа: <https://arxiv.org/abs/1910.06764>, свободный (дата обращения: 30.03.2026).
- [13] Perez, E., Strub, F., de Vries, H. et al. FiLM: Visual Reasoning with a General Conditioning Layer // Proceedings of AAAI. — 2018. — Режим доступа: <https://arxiv.org/abs/1709.07871>, свободный (дата обращения: 30.03.2026).
- [14] Ren, S. Reinforcement Learning for Hearthstone Battlegrounds // Leiden University, LIACS. — 2022. — Режим доступа: <https://theses.liacs.nl/2366>, свободный (дата обращения: 30.03.2026).
- [15] Rybkin, O. et al. Scale Anchoring for Stable PPO Training with Transformers // arXiv preprint. — 2026. — arXiv:2602.10408. — Режим доступа: <https://arxiv.org/abs/2602.10408>, свободный (дата обращения: 30.03.2026).
- [16] Schrittwieser, J., Antonoglou, I., Hubert, T. et al. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model // Nature. — 2020. — Vol. 588. — P. 604–609. — Режим доступа: <https://arxiv.org/abs/1911.08265>, свободный (дата обращения: 30.03.2026).
- [17] Shazeer, N. GLU Variants Improve Transformer // arXiv preprint. — 2020. — arXiv:2002.05202. — Режим доступа: <https://arxiv.org/abs/2002.05202>, свободный (дата обращения: 30.03.2026).
- [18] Vieira, R., Tavares, A. R., Chaimowicz, L. Exploring Deep Reinforcement Learning for Battling in Collectible Card Games // Proceedings of SBGames. — 2022. — Режим доступа: <https://doi.org/10.1109/SBGAMES56371.2022.9961110>, свободный (дата обращения: 30.03.2026).
- [19] Vinyals, O., Babuschkin, I., Czarnecki, W. M. et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning // Nature. — 2019. — Vol. 575. — P. 350–354. — Режим доступа: <https://www.nature.com/articles/s41586-019-1724-z>, свободный (дата обращения: 30.03.2026).
- [20] Vinyals, O., Fortunato, M., Jaitly, N. Pointer Networks // Proceedings of NeurIPS. — 2015. — arXiv:1506.03134. — Режим доступа: <https://arxiv.org/abs/1506.03134>, свободный (дата обращения: 30.03.2026).
- [21] Wen, C. et al. SAINT: Scalable Autoregressive Interaction Network for Transformer Agents // arXiv preprint. — 2025. — arXiv:2505.12109. — Режим доступа: <https://arxiv.org/abs/2505.12109>, свободный (дата обращения: 30.03.2026).

- [22] Xiong, R., Yang, Y., He, D. et al. On Layer Normalization in the Transformer Architecture // Proceedings of ICML. — 2020. — arXiv:2002.04745. — Режим доступа: <https://arxiv.org/abs/2002.04745>, свободный (дата обращения: 30.03.2026).
- [23] Zhang, B., Sennrich, R. Root Mean Square Layer Normalization // Proceedings of NeurIPS. — 2019. — arXiv:1910.07467. — Режим доступа: <https://arxiv.org/abs/1910.07467>, свободный (дата обращения: 30.03.2026).
- [24] Zhang, W. Design and Implementation of an Intelligent Agent for Auto Battler Games Using Reinforcement Learning // Bachelor's Thesis, Lappeenranta–Lahti University of Technology. — 2024. — Режим доступа: <https://lutpub.lut.fi/handle/10024/167789>, свободный (дата обращения: 30.03.2026).
- [25] Zheng, Q. et al. Online Decision Transformer // arXiv preprint. — 2022. — arXiv:2202.05607. — Режим доступа: <https://arxiv.org/abs/2202.05607>, свободный (дата обращения: 30.03.2026).